TRAVELING SALESMAN PROBLEM DEMONSTRATION

DAA Mini Project

Programming Language: Python 3

1. Problem Statement

The Traveling Salesman Problem (TSP) asks for a minimum-cost tour through a set of cities. The tour starts at a chosen city, visits every other city exactly once, and returns to the start. This project demonstrates exact and heuristic solutions and compares their efficiency.

2. Objectives

Model TSP as a weighted graph; implement exhaustive search, Dynamic Programming and a greedy heuristic; establish correctness; analyze time and space complexity; and measure practical execution time.

3. Algorithm Design

Brute Force: Fix the first city, generate all permutations of the remaining cities, calculate every tour, and retain the minimum.

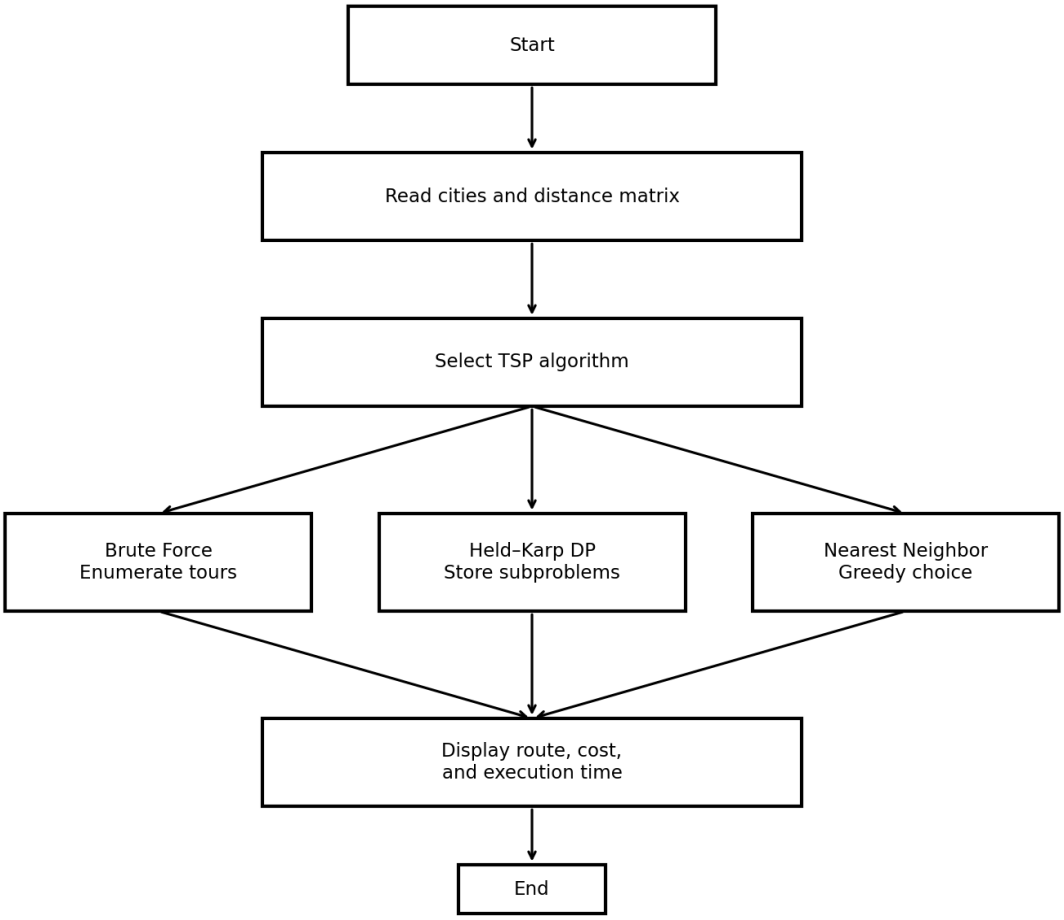
Held–Karp: Define dp[mask][i] as the minimum cost from city 0 through the cities in mask ending at i. Transition by trying every possible predecessor. Finally add the edge back to city 0.

Nearest Neighbor: Repeatedly select the nearest unvisited city and close the cycle at the end.

4. Correctness

Brute Force is correct because every possible tour is enumerated. Held–Karp is correct because each DP state considers all possible predecessors and uses optimal substructure; the minimum final state plus the return edge is globally optimal. Nearest Neighbor correctly implements its greedy rule and produces a valid tour, but does not guarantee optimality.

5. Flowchart



6. Technologies / Programming Language

Python 3.10+ and standard-library modules. Automated tests use unittest and runtime measurement uses time.perf_counter(). No external runtime dependencies are required.

7. Complexity Analysis

Algorithm	Time	Space	Optimal?
Brute Force	O((n–1)!)	O(n) auxiliary	Yes
Held–Karp DP	O(n² × 2■)	O(n × 2■)	Yes
Nearest Neighbor	O(n²)	O(n)	No

The factorial growth of exhaustive search makes it impractical for larger n. Held–Karp avoids repeated subproblem computation and reduces the exact solution to exponential time. Nearest Neighbor has polynomial time but sacrifices the guarantee of optimality.

8. Test Cases and Output

Test	Condition	Expected Result
TC01	4-city matrix	Exact cost = 80
TC02	Nearest Neighbor sample	Valid closed tour
TC03	Single city	Cost 0, route 0 → 0
TC04	Random 8-city graph	All algorithms return closed tours
TC05	Invalid matrix	Validation error

Sample input:

0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

The optimal tour cost is 80.

9. Output Screenshots

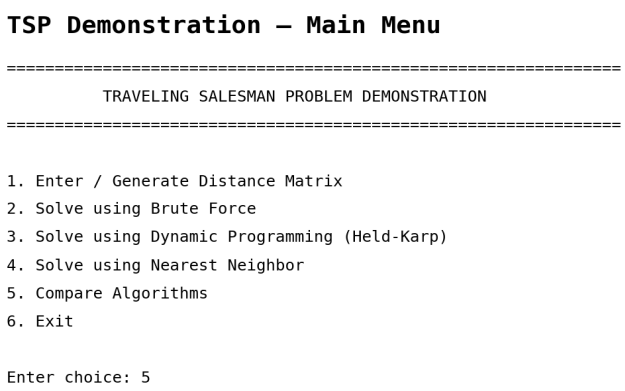


Figure 1. Main menu

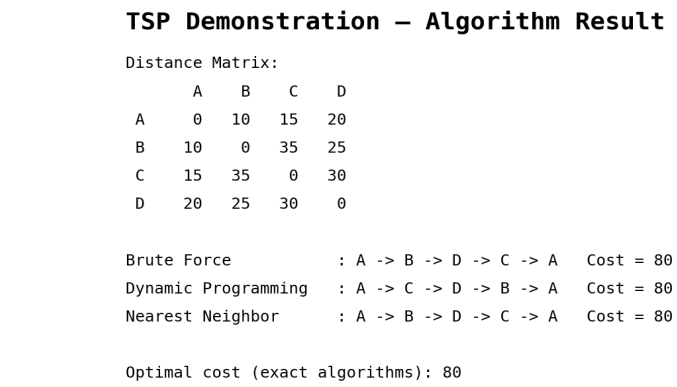


Figure 2. Algorithm result

TSP Demonstration — Performance Comparison

PERFORMANCE COMPARISON

```
Cities   Algorithm                Cost      Median Time (s)
4        Brute Force                109           0.00000367
4        Dynamic Programming        109           0.00000754
4        Nearest Neighbor           109           0.00000141
6        Brute Force                244           0.00006761
6        Dynamic Programming        244           0.00005019
6        Nearest Neighbor           257           0.00000235
8        Brute Force                261           0.00388768
8        Dynamic Programming        261           0.00034162
8        Nearest Neighbor           301           0.00000359
10       Brute Force                320           0.31665939
10       Dynamic Programming        320           0.00279176
10       Nearest Neighbor           344           0.00000501
```

Figure 3. Performance comparison

10. Performance Comparison

Cities	Algorithm	Cost	Median Time (sec)
4	Brute Force	109	0.00000367
4	Dynamic Programming	109	0.00000754
4	Nearest Neighbor	109	0.00000141
6	Brute Force	244	0.00006761
6	Dynamic Programming	244	0.00005019
6	Nearest Neighbor	257	0.00000235
8	Brute Force	261	0.00388768
8	Dynamic Programming	261	0.00034162
8	Nearest Neighbor	301	0.00000359
10	Brute Force	320	0.31665939
10	Dynamic Programming	320	0.00279176
10	Nearest Neighbor	344	0.00000501

Measured runtime depends on hardware and runtime conditions; asymptotic complexity is the primary theoretical comparison.

11. Conclusion

The project connects graph modeling, exhaustive search, Dynamic Programming optimization, greedy heuristics, correctness, complexity analysis, testing and practical benchmarking. The three methods make the trade-off between solution quality and computational cost visible.

12. Future Scope

Branch and Bound, 2-opt local search, graphical visualization, benchmark plots, asymmetric TSP and real map data can be added.

13. Repository / Execution

Run with **python tsp.py**. Test with **python -m unittest discover -s tests -v**. The GitHub repository should contain all source files, README.md, screenshots and this PDF report.